

doc-ai-kit

A framework for AI-led document processing at scale

September 2026

The job

You have a set of documents and the same questions to answer about each one, and there are too many of them for an analyst to read.

Claude can read the documents and answer those questions well enough to be useful. Even so, a substantial amount of work remains for the analyst who must:

- Choose a subset of documents and divide it into training, validation and test sets.
- Plan the labeling, then do it.
- Write the code that reads the documents and pulls out their text.
- Design the prompts, and write the code that sends them with the document text.
- Measure how well the answers do, then change the prompts and measure again.
- Write the code that runs the final prompts over the whole corpus.
- Collect the results and write a report.

None of it is difficult on its own. It is a week or two of setup before anyone sees a number, and it is rebuilt from scratch on the next project.

Most of that can be handed to an agent

An agent can do this work, given the right tools and the right instructions. doc-ai-kit supplies both.

The tools

A Python package, installed with pip at a fixed version. It provides the commands for each step: extracting text, drawing the sets, building the labeling sheet, scoring, tuning, running the corpus, writing the report.

A second command, `newproject`, creates a project folder with the settings, the file where the questions are declared, and somewhere for project-specific code.

The instructions

The new project also contains written guidance for each step: what it is for, what to run, what the output means, what to decide, and what comes next.

A Claude Code session started in the project folder reads that guidance and works through the project with you.

The commands work on their own as well. The guidance tells the agent how to use them, and where to stop and ask.

What the agent does with you

- Tells you where to put your documents and how to organise them.
- Helps you settle the questions to be answered and the form each answer takes.
- Writes the configuration and the first version of the prompts.
- Draws the training, validation and test sets, and explains what each is for.
- Produces a spreadsheet for you to label, and reads it back once you have filled it in.
- Checks for answers that appear too rarely to measure, and talks through the options.
- Runs the prompt tuning, one experiment at a time, within the limits you set.
- Applies the best prompt to every document and produces the answers.
- Writes the final report, including how accurate each answer is.

The labeling stays with you. Everything else is a conversation.

Value proposition

- Something that took weeks now takes hours
- Provides a repeatable and traceable process
- Allows enough flexibility to handle novel problems
- Helps to prevent scope creep and excessive iteration
- Ensures consistency across projects

Inside a project

What you and the agent fill in

- `tasks.yaml` : the questions, the form of each answer, and how answers are compared
- `config.yaml` : the model, the limits, the spend ceiling
- `custom/` : project code, registered rather than patched into the package
- `data/pdfs/` : the documents

What the project accumulates

- `data/text/` : the extracted text, read by every later step
- `data/labels.xlsx` and `labels.jsonl` : your labels, before and after checking
- `data/splits.json` : which documents are training, validation and held back
- `runs/` : every scoring run, the experiment leaderboard, the spend
- `decisions.md` : each decision you made, with the numbers behind it
- `REPORT.md` and `PROMPT.md` : written at the end

The path a document takes

```
pdfs -> text cache -> DSPy program -> typed answers -> normalize -> compare
                        ^                                     |
                        |                                     v
                    optimizer <----- score <----- human labels
```

- Text is extracted once and cached. Tuning and production read the same text, because accuracy measured on one version of a document does not carry over to another.
- Answers come back typed: a date as a date, a number with its unit, a list as a list.
- Normalizing before comparing is what makes "\$1.25M" and "1,250,000 USD" the same answer, and "Texas" and "TX" the same answer.
- The score is what the optimizer uses to choose between one prompt and another.

Nine kinds of question

KIND	EXAMPLE	HOW AN ANSWER IS JUDGED
Yes or no	Is liability capped?	Same answer, reading yes/no loosely
One of a list	Which category of document is it?	Same value, after mapping synonyms to one spelling
Several of a list	Which risk flags apply?	The two sets compared
Exact text	The reference number	Equal after tidying spacing and case
Close-enough text	The organisation's name	Similar enough, above a threshold the project sets
A list of values	Who signed it?	Each answer paired with its best match; extras and misses counted
A number with a unit	The total amount, or a notice period	Same unit, within the project's tolerance
A date	The date it takes effect	Equal at the precision the document gave
A passage	The paragraph that states the obligation	Enough overlap with the labeled passage

The kind of question sets the comparison, the type of the answer, and the numbers the report can show. A fixed-choice question asked as free text loses the per-answer breakdown.

Scoring

- The label and the model's answer pass through the same cleanup code before they are compared, so the comparison cannot favour either side.
- A comparison returns counts, not a verdict: correct answers, wrong answers, missed answers, correct blanks. The report needs those separately to show how often the program is right apart from how much it finds.
- A wrong answer counts twice, as something got wrong and something missed.
- A blank against a blank label counts as correct. Most documents do not answer every question.
- A reply that cannot be parsed counts as a failure rather than a blank. It is retried, and by default the run stops rather than report numbers if it still fails.
- The comparison code has its own test set of near-misses, and nothing downstream runs until those pass.

Three sets of documents

Training

Supplies the worked examples that go into the prompt.

Validation

Used to compare versions of the prompt and choose one.

Held back

Scored once, at the end. That number is the one reported.

- The held-back documents are chosen before any model runs, from a seed recorded in the project.
- They are labeled without looking at model output. Labels made by correcting the model agree with the model, and any number measured against them comes out too high.
- They are scored once. A second scoring needs an explicit override, and the override is printed in the report.
- Questions with too few examples among them are reported as unmeasured rather than given a number.

Labeling

Labels come from a spreadsheet: one tab, one row per file, one column per question.

- `sample-labels` chooses which documents to label, and which of them are held back, before any model runs.
- `label-sheet` writes the spreadsheet. Rows can arrive filled in with the model's answers for correction, which is two to three times faster, or empty. The labeler chooses. Held-back rows are always empty.
- Answers are written the way people write them: `$1.25M`, `June 15, 2024`, `30 days`. A passage is labeled by pasting the sentence, and doc-ai-kit finds its position in the text.
- `import-labels` reads every cell with the scoring code, reports all problems at once, and writes nothing until they are fixed. It also warns when one cell holds two entries, which cannot be matched and lowers that question's ceiling.

How tuning works

The model is not retrained. What changes is the message it receives: some instructions, the questions from `tasks.yaml`, sometimes a few worked examples, then the document. DSPy, the open-source library doing the tuning, tries versions of that message, scores each one on the validation set, and keeps the best.

BootstrapFewShot

Runs the program, keeps the documents it answered entirely correctly, and puts a few in the prompt as examples. Every answer category gets at least one.

MIPROv2

Has a model draft alternative instructions, then tries combinations of instructions and examples, spending more attempts on the ones that score well.

GEPA

Reads the mistakes, including the question, the expected answer and what came back, and has a second model rewrite the instructions.

Each experiment changes one thing and records its score per question, its cost and its runtime. The best so far is pinned, and the next experiment starts from it.

Limits enforced in code

LIMIT	WHAT IT PREVENTS
The held-back set locks after one scoring	Scoring twice and keeping the better number
Labels and splits are read-only during tuning	A run editing the thing it is measured against
A set number of experiments and model calls	A search that continues until the validation set gives way
A spend ceiling, checked before each paid step	Learning the cost after the fact. Spend is recorded per step
Tuning requires written labeling rules	A question whose rule was never written down, and so cannot be labeled consistently
Replies are parsed strictly	An unreadable reply counting as a blank

Each refusal says what to change. Limits can be raised, but only as a decision recorded in the project's log.

Traceability and repeatability

Every run records what produced it

- the settings: optimizer, model, call limits, the size of each set
- the one thing that experiment changed, named by the person who ran it
- which experiment it started from
- its score per question, its cost and its runtime
- the package version and a fingerprint of the settings
- the prompt itself, loaded unchanged for the full run

A leaderboard holds one row per experiment. `decisions.md` holds the choices a person made, with the numbers they had at the time.

The model itself is not repeatable: the same prompt can come back worded differently. The procedure, the sets and the scoring are fixed, which is what makes a difference measurable.

The process is the same every project

- the same commands in the same order, from one package version
- the sets drawn from a recorded seed, so the division can be reproduced
- text extracted once and cached, so every later step reads the same words
- answers saved from every run, so scoring can be corrected and repeated without calling the model again
- one row per finished project in a shared ledger

Take any number in the report, open the run behind it, and the prompt, the settings and the documents are all there.

What the report shows

- Per-question numbers appear next to the average in every run, because the average on its own hides a question that has collapsed.
- Yes/no questions are reported on their yes answers, which is what the headline number measures. Counting both answers together raises it: a program that finds 6 of the 14 documents where the answer is yes, and answers no correctly everywhere else, reads 0.75 either way when both answers are pooled, against 1.00 precision and 0.43 recall on the yes answers.
- Every number carries the range it could sit in. Ten documents, all correct, still allow a true error rate near 25%.
- The error file lists three mistakes per question. Given the full list, the optimizer writes rules about individual documents that do not hold on new ones.
- Borderline matches are listed for review, raw and normalized, because no threshold separates a typo from a different company.
- Predictions are saved, so correcting the comparison code and re-scoring costs nothing.

Scanned pages

A scanned page has no text layer, so any question answered from it fails, and no change to the prompt affects that.

- Pages are judged one at a time. A page with almost no text, where images cover most of it, is sent to Claude as an image, and the transcription replaces that page in the text cache.
- Judging a document on its average misses this. A report with twenty typed pages and five scanned pages of accounts averages out fine, and those five pages come through empty.
- Tables come back row by row, so a figure stays in its row. About 3,900 input tokens per page. Each page is cached, so re-extracting does not pay for it again.
- Pages that still have no text are counted in the manifest, and those documents are sent for review.

Running the full corpus

How it runs

Production goes through Anthropic's batch service at half the price. The requests are captured from the normal path at the point of sending, so a batch request is the same request, and replies are parsed by the same code.

Measured on a test corpus: 48 of 48 requests succeeded, input tokens matched the live run exactly, accuracy was 0.996 either way, and the cost went from \$0.238 to \$0.119.

Batch identifiers are written down before any waiting, and each answer is saved as it arrives, so an interrupted run resumes instead of paying twice.

Checks before handover

- every document produced an answer, and failures are recorded rather than dropped
- replies that could not be parsed stay under a set share
- the spread of answers matches the labeled sample
- blanks are not far above what the validation set showed

Documents are sent for review for five reasons, one of which is a random sample. The other four select documents that are already doubtful, so a quality estimate from those alone reads worse than the run is.

Testing the comparison code

If the comparison is wrong, every number after it is wrong, and the search spends money working against it.

- A generated corpus, contracts in this case, where a plausible wrong answer sits near the right one: a former company name, a second dollar figure, a notary in the signature block, a venue clause naming a different state from the governing-law clause.
- Each case lists the forms a correct answer might take, which the comparison must accept, and the near-miss it must reject.
- It runs in the test suite without model calls. It found four faults in the comparison code: dotted legal suffixes, titles on personal names, currency written as a word, and a typed answer that skipped unit handling.
- One limit is recorded rather than fixed. String similarity cannot separate a typo from a different company, so "Smith & Hartley Ltd." is accepted for "Smith and Hart Limited", and borderline cases go to review.

The commands

COMMAND	WHAT IT DOES
<code>newproject</code>	Create the project, pinned to one version of the package
<code>extract</code>	PDFs to cached text, scanned pages transcribed
<code>sample-labels</code> , <code>label-sheet</code> , <code>import-labels</code>	Choose what to label, produce the spreadsheet, read it back
<code>audit-labels</code>	Check labels against the questions and count examples per answer
<code>make-splits</code>	Divide the labeled documents; stop where an answer is too rare to measure
<code>run-baseline</code>	Score simple versions, so tuning has something to beat
<code>compile</code>	One tuning experiment, changing one thing
<code>holdout</code>	Score the held-back documents, once
<code>production</code>	Run the whole corpus and check the output
<code>rescore</code> , <code>adjudicate</code>	Re-score a finished run at no cost; list the borderline matches
<code>close</code>	Write the report, the shipped prompt, and the ledger row

`status` works out where a project stands from what is on disk, so it is correct in a new session. Each phase has a guidance file that a Claude Code session reads before running it.

What a finished project hands over

FILE	CONTENTS
<code>REPORT.md</code>	Written for people who did not build the project: what it does, how well it works, what still needs a person. Each number links to the file it came from
<code>PROMPT.md</code>	The prompt that shipped, in readable form: instructions, the question per task, the worked examples
<code>outputs.jsonl</code>	The answers for every document, including failures
<code>qa_report.md</code>	The checks on the full run and the documents queued for review
<code>decisions.md</code>	Each decision a person made, with the numbers it was made on
<code>spend.json</code>	What each paid step cost
<code>ledger.csv</code>	One row per project, so the next one can be estimated from it

The report is regenerated each time `close` runs. Anything written by hand goes in `REPORT_NOTES.md`, which is appended to it.

What the measurement does not cover

- Small sets give wide ranges. Thirty documents cannot settle a question to within a few points, and the report gives the range rather than implying precision.
- Rare answers are reported as unmeasured. At 2% prevalence, reaching thirty examples by random sampling takes about 1,500 documents. The options are to label more of that kind, merge it with a neighbouring answer, or leave it unmeasured, and that choice is recorded.
- Answers come from text. Scanned pages are transcribed first, so page layout is not available to the model when it answers.
- A labeling mistake looks the same as a model mistake. Reading the failures by hand is part of the work.
- Running a real project found fourteen bugs in the package, one of which paid for a batch and then paid again for live requests. All are fixed, and each has a test.

Starting a project

```
newproject "Supplier Agreements"    # creates the project, pinned to one version
cd supplier-agreements && make lock && make sync
claude                             # from inside the project folder
```

Claude then runs the commands, explains what comes back, and stops at the decisions that are yours.

- Yours to decide: how many documents to label, whether the spreadsheet arrives filled in, what to do about an answer with too few examples, whether to ship the tuned prompt or the simple one, and when to score the held-back documents.
- Not Claude's to do: the labels, and in particular the held-back ones. Labels written by a model are not human labels, and there they would measure the model against itself.

The project's `README.md` and `docs/protocol.md` explain each step.

A real project, screen by screen

The rest of the deck is one project as it actually ran: about 500 commercial contracts, nine questions, from an empty folder to a finished report.

The screenshots were taken while the package was still called `doc-harness` .

Day one: where to begin

The session opens by checking what is on disk and saying what to do next.

What to do now

Step 1: Put your documents in data/pdfs/. Everything else works on these files.

Step 2: Decide what to pull out of each document and write it in tasks.yaml. Each task is one question per document, such as "the contract number" or "does it have an arbitration clause". The file has one commented example for each of the nine task types. Uncomment the ones you want, edit the questions and delete the rest. I can help with this if you tell me what the documents are and what you want from them.

Step 3: Set the models in config.yaml. For example:

models:

task: anthropic/claude-sonnet-5

reflection: anthropic/claude-sonnet-5

Setting a model means runs will cost real money, so this choice is yours. If some PDFs are scans, also set extraction.transcription.model.

After that

Run `doc-harness status` after each step. It works out where you are from the files on disk. The remaining steps each have a skill (slash command), in this order:

`/extract` → `/sample-labels` → `/label-sheet` (you label documents) → `/import-labels` → `/audit-labels` → `/make-splits` → `/run-baseline` → `/compile` → `/holdout` → `/production` → `/close`

Two rules to keep in mind from the start:

- **You get one look at the holdout set.** `/holdout` can only be run once.
- **Label holdout documents without seeing the model's answers.** You'll label them from scratch.

It's also worth reading `docs/protocol.md` once before you start labeling.

Question for you: what kind of documents are these, and what do you want to pull out of them? With that I can draft `tasks.yaml` for you.

* Baked for 16s · done Tuesday, Sep 22 at 10:18 AM

› Ok I did step 1

The questions, in plain English

The questions were written as a list in ordinary words, not as configuration.

```
What do you want to learn from these contracts? Pick from the list above or tell me your own, and I'll write tasks.yaml for you.
```

```
* Churned for 32s · done Tuesday, Sep 22 at 10:20 AM
```

```
* recap: You're setting up a harness to pull answers out of 510 contracts, and the PDFs are now in place. Next, tell me which 4–6 questions you want answered for each contract, and I'll write tasks.yaml. (disable recaps in /config)
```

```
> | I have about 500 commercial contracts in PDF. For each one, I'd like to know:
```

```
| 1. What kind of agreement it is: distribution, license, supply, services, joint venture, franchise  
| and so on. One category per contract.
```

```
| 2. Who the parties are: the company names, not the people who signed.
```

```
| 3. When it was signed. Sometimes only the year or month is given.
```

```
| 4. Which state's law governs it. Mostly US states, but some are foreign.
```

```
| 5. Whether it restricts either side from competing, and if so, which kinds of restriction: a  
| non-compete, exclusivity, no poaching of customers, no poaching of employees. It can have several  
| or none.
```

```
| 6. Whether there's a cap on liability. If there is, I also want the actual clause text, so a lawyer  
| can read it.
```

```
| 7. Whether either party can audit the other's books or records.
```

```
| 8. If the contract renews automatically, how much notice is needed to stop it renewing, e.g. 30 or  
| 90 days.
```

```
| 9. Whether either side can end it early just by giving notice, without needing a reason.
```

```
| I'd rather get a blank than a guess when the contract doesn't say.
```

```
----
```

The questions, turned into tasks

Nine questions became ten tasks, each with a type and a rule for when the answer is blank.

- I wrote `tasks.yaml` and the harness reads it without errors. Status has moved to **tasks declared**. Your 9 questions became 10 tasks, because #6 is split into a yes/no task and a task that pulls out the clause text.

#	Task	Type	Blank when...
1	<code>agreement_type</code>	one of 25 categories	never; every contract gets one
2	<code>parties</code>	list of company names	no party is named
3	<code>agreement_date</code>	date, can be year-only or month-only	no date is given
4	<code>governing_law</code>	state code, or FOREIGN	no governing law is named
5	<code>competition_restrictions</code>	any of 4 labels	an empty list means none
6a	<code>has_liability_cap</code>	yes/no	never blank
6b	<code>liability_cap_clause</code>	the clause text	there is no cap
7	<code>has_audit_rights</code>	yes/no	never blank
8	<code>renewal_notice_days</code>	number of days	it doesn't auto-renew, or no notice is stated
9	<code>termination_for_convenience</code>	yes/no	never blank

Choices I made that you should check

- **Yes/no questions never come back blank**. If a contract has no audit clause, the answer is "no", not "unknown". Otherwise labelers would split between blank and "no" for the same situation, and the scores would get noisy. Your "blank rather than guess" rule applies to the other tasks: dates, law, parties, notice period and the clause text.

Getting the text out

Text extraction is free. Two pages were scans; the rest had a text layer.

```
Should I run /extract?

* Crunched for 29s · done Tuesday, Sep 22 at 10:28 AM

> yeah run it

● Skill(/extract)
  | Successfully loaded skill

Ran 1 shell command

● All 509 extracted. Two documents have one scanned page each. Checking the manifest, the text quality and the document lengths.

Ran 2 shell commands

● The text is clean. Now checking what's on the two scanned pages, to see whether they matter.

Read 4 files, ran 3 shell commands

● Extraction is done, and it cost nothing.

  - All 509 contracts are now text files in data/text/. I checked a few samples and the text is clean.
  - Every contract fits in Haiku's context. The longest is about 87K tokens and the limit is 200K, so nothing needed cutting.
```

How many documents to label

Before labeling, how often each answer appears, and how many examples each question will get.

yeah recommend a number

I estimated how common each answer is. I used only file names and simple word searches over all 509 contracts, so treat these as rough guides.

Task	Rough share of contracts	Expected in the 150 non-holdout docs (if you label 200)
Liability cap = yes	~18%	~27
Termination for convenience = yes	~15%	~22
Auto-renews (so has a notice period)	~12%	~18
Non-compete	~13%	~20
No-solicit of customers	~7%	~10
Governing law: NY / CA / DE	~11% / 6% / 5%	~17 / 9 / 7
Each agreement type	2-9% each	~3-14 each

The harness wants **30 examples** of a class before it will tune the model for that class (`support_floor`). With fewer, the class is still scored and reported, just not tuned. With fewer than 10, it gets no reported number at all.

Why 200

- **200 is the cutoff for the best split.** At 200 or more, the harness splits 50/25/25: 100 to train, 50 for validation, and 50 for the holdout. Below 200 it keeps a bigger share (30%) for the holdout.
- **The common yes/no questions get close to 30.** Liability cap and termination for convenience land near the floor. Going higher helps them, but slowly.
- **Nothing reaches the rare ones.** Look at no-solicit of customers, most states and most agreement types. Even labeling all 509 contracts gives most agreement types fewer than 30 examples. The largest type has about 46 in the whole corpus. Labeling more doesn't fix that. Merging categories and adding targeted examples do, and both are decided later at `/make-splits`.

What labeling will cost in time

Three sizes, the hours each implies, and the decisions to settle first.

> yeah recommend a number

Count	Holdout (labeled from scratch)	Others (you correct the model's answers)	Rough total
120	36	84	~23 hours
200	50	150	~37 hours
300	75	225	~55 hours

(These assume ~20 minutes per contract labeled from scratch and ~8 minutes per corrected one.)

Decide now: merge the 25 agreement types

You'll almost certainly need to merge them into about 6–8 groups, for example putting reseller with distributor, and consulting with service. It's easier to do that in `tasks.yaml` **before** labeling than to redo labels later. I can propose a grouping.

Other option: if you plan to use the existing CUAD answer key instead of labeling by hand, labeling effort no longer limits you and all 509 could be labeled. That would be a labeling decision to record.

Shall I run `doc-harness sample-labels --count 200`? Or should I propose the agreement-type grouping first?

A decision made before labeling, not after

Twenty-five contract types were merged into seven, because merging later means relabeling.

The draw is now fixed. Once any labels exist, the harness refuses to redraw it.

Before labeling: agreement types

Decide this before you start labeling. `agreement_type` has 25 categories. Even in all 509 contracts, most types have fewer than 30 examples. In 84 labeled documents, most will have 0–5, too few for any of them to get a reported score. If you merge them into about 6–8 groups now, the labels come out usable. Merging after labeling means going back over those labels.

Here is a possible grouping:

- **Distribution & resale:** distributor, reseller, agency
- **IP & licensing:** license, ip, franchise
- **Services:** service, consulting, maintenance, outsourcing, hosting, transportation
- **Supply & manufacturing:** supply, manufacturing
- **Marketing:** marketing, promotion, co_branding, sponsorship, endorsement, affiliate
- **Partnership:** joint_venture, strategic_alliance, collaboration, development
- **Non-compete:** non_compete

Next: /label-sheet

This writes `data/labels.xlsx` and pre-fills the 84 non-holdout rows with the model's answers, for about \$1.50. It's the first step that spends money.

Should I merge the agreement types as above (or your version) before building the sheet? Or keep all 25?

* Brewed for 19s · done Tuesday, Sep 22 at 11:58 AM

> merge them as you proposed, then build the sheet

The labeling sheet

One spreadsheet, one row per contract, with what each column wants and in what format.

```
> merge them as you proposed, then build the sheet
```

- The sheet is ready: `data/labels.xlsx`, 120 rows, one per contract. No model was run, so it cost \$0. Every row starts empty and all of them are shaded. That's expected: without prefill, every label is your own reading. I logged the choice in `decisions.md`.

Open each contract from `data/pdfs/` using the name in `file_name`, and fill in the row. Hovering over a column header also shows its question and format.

What each column wants

Column: `agreement_type`

What to write: The main purpose of the contract. Go by what it does, not just its title. An amendment gets the type of the contract it amends.

Format: Pick one from the dropdown: `distribution`, `ip_licensing`, `services`, `supply_manufacturing`, `marketing`, `partnership`, `non_compete`. Never blank.

Column: `parties`

What to write: The `companies` that are parties to the contract. Leave out the people who signed, and leave out lawyers or affiliates unless they're named as parties. A person counts only if they personally are a party.

Format: Names as written, separated by semicolons: `Acme Inc.; Beta LLC`

Column: `agreement_date`

What to write: The date it was signed or is dated.

Format: `2019-06-15` or `June 15, 2019`. If only the month or year is given, write just that (`2019-06` or `2019`). Never fill in a missing part. Blank if there's no date.

Importing and checking the labels

The import checks every cell, then the labels themselves are read for mistakes.

```
› All good. Labels are done now.  
  └ 1 skill available  
  
● Skill(/import-labels)  
  └ Successfully loaded skill  
  
Ran 1 shell command  
  
● Imported cleanly: 120 documents, all blind, including all 36 holdout rows. Running the audit.  
  
● Skill(/audit-labels)  
  └ Successfully loaded skill  
  
Ran 3 shell commands  
  
● The audit passes, but let me check the label values themselves. The first parties row looks wrong.  
  
Ran 1 shell command  
  
● Found a labeling problem. Checking the other columns for consistency too.  
  
Ran 1 shell command  
  
● The audit passes: no unknown tasks, no bad values, all 120 blind. Two things need your attention.
```

The starting point, per question

The first scored run, before any tuning, with a reading for each question.

- Both baselines scored the same, as expected. But `governing_law` came out at 0.126 – near zero, which the protocol says to diagnose before spending any optimization budget.

- `Skill(reading-failures)`
 - └ Successfully loaded skill

Ran 2 shell commands

- Good news: `governing_law` isn't broken. Let me verify one thing about how the optimizer will see it.

Ran 2 shell commands

- Baselines recorded. No failed replies. **Best baseline: zero-shot at 0.823 aggregate.** BootstrapFewShot scored identically, as expected with demos set to 0, so the real bar is zero-shot.

Task	Score	Reading
has_audit_rights	0.933	strong
parties	0.851	at its ceiling of ~0.92 already
agreement_date	0.844	strong
termination_for_convenience	0.846	strong
has_liability_cap	0.818	solid
competition_restrictions	0.789	solid
renewal_notice_days	0.750	only 4 examples; don't trust it
liability_cap_clause	0.627	real weakness
agreement_type	0.543	see below
governing_law	0.126	not broken – see below

Tuning, and when to stop

What the run gained, what it lost, what has been spent, and the choice of whether to spend more.

What moved: chain-of-thought lifted five tasks, all on the main predictor, by 5–13 points each. That's a consistent pattern across several tasks, not one document moving one number, so I'd treat it as real – though on 25 documents each point is a quarter of a document, so the exact values aren't.

What it lost: exp_001's spans gain (0.693 → 0.628). That instruction lived in the `predict` program, and exp_002 started fresh, so it's gone. The two best results are currently in different programs and can't be merged: the CLI has no way to carry one instruction across.

`renewal_notice_days` dropped, but it has 4 examples – ignore it. `agreement_type` and `governing_law` are the diluted macro scores, unchanged.

Spend: roughly \$24 of \$75. One experiment left.

Two ways to finish:

1. **Stop and go to the holdout now.** Ship exp_002. Remaining cost about \$15 (holdout ~\$3, production ~\$12 with chain-of-thought), landing near \$40 total.
2. **Use the last experiment** to run MIPROv2 on top of exp_002, which is the one thing that plausibly recovers the spans instruction under chain-of-thought. About \$12, and it may well land inside the noise band.

I lean toward 1: you already have a clear winner over the baseline, and the remaining headroom on spans is partly label noise we've already measured. Which do you want?

The final test

The set-aside documents, scored once. Some questions held up; two did not.

- The holdout is spent and locked. Here are the results.

Aggregate: 0.822 (95% CI 0.766–0.874) against 0.873 on validation – a gap of 0.051, which sits inside the confidence interval.

Task	Holdout	Validation	Gap	Reading
agreement_date	0.935	0.913	−0.02	solid
parties	0.911	0.855	−0.06	solid, and above the 0.92 label ceiling in practice
has_audit_rights	0.889	1.000	+0.11	good
agreement_type	0.657	0.531	−0.13	better than validation
competition_restrictions	0.696	0.919	+0.22	validation was optimistic
termination_for_convenience	0.667	0.917	+0.25	validation was optimistic
has_liability_cap	0.600	0.870	+0.27	validation was optimistic
liability_cap_clause	0.234	0.628	+0.40	collapsed
governing_law	0.173 macro	0.126	−	actually perfect: P 1.00, R 1.00 on 29 documents
renewal_notice_days	1.000	0.667	−	only 7 examples; ignore

The overall gap is normal. 5 points on 32 documents, well within the interval. The champion generalized.

Reading the numbers honestly

Writing the report found a question whose score looked fine and was not.

- Rewritten and pushed. The report is now ~1,800 words of plain English, with every link and anchor checked.

What changed in substance, not just wording:

The rewrite turned up a real problem the old report hid. For yes/no questions, the harness's headline "F1" counts correct *no* answers too, so it reads as plain accuracy. `has_liability_cap` showed **0.750**, which sounds fine – but the model found only **6 of the 14** contracts that actually have a cap. It just happened to be right about all the "no" answers.

So **I need to correct something I told you earlier**. I said about one cap in four is missed. It's more than half. The report now says so, and `decisions.md` records the correction.

How the report reads now:

- Results are **counts out of 32**, not scores: "Right on all 29 contracts that name one", "Found 6 of the 14".
- Answers are grouped by what you should *do* with them: **trustworthy** (governing law, dates, parties, renewal notice), **usable with a check** (agreement type, audit rights, early termination, competition restrictions), **needs a person** (both liability cap fields).
- That confusing paragraph now reads: *"The other groups are contracts we already suspect are wrong, so they will look worse than the corpus as a whole. If you only check those, you will think the system is worse than it is."*
- Known problems are numbered and explained in ordinary words, including why two scores (0.657 and 0.173) look terrible but aren't.
- There's a glossary for anyone who opens the technical files, and a caution that one contract is worth 3 points.